

Scaler Business Case Study:- Target SQL

Context:

Target is a globally renowned brand and a prominent retailer in the United States. Target makes itself a preferred shopping destination by offering outstanding value, inspiration, innovation and an exceptional guest experience that no other retailer can deliver.

This particular business case focuses on the operations of Target in Brazil and provides insightful information about 100,000 orders placed between 2016 and 2018. The dataset offers a comprehensive view of various dimensions including the order status, price, payment and freight performance, customer location, product attributes, and customer reviews.

By analyzing this extensive dataset, it becomes possible to gain valuable insights into Target's operations in Brazil. The information can shed light on various aspects of the business, such as order processing, pricing strategies, payment and shipping efficiency, customer demographics, product characteristics, and customer satisfaction levels.

The data is available in 8 csv files:

- 1.customers.csv
- 2.sellers.csv
- 3.order_items.csv
- 4.geolocation.csv
- 5.payments.csv
- 6.reviews.csv
- 7.orders.csv
- 8.products.csv

Problem Statement:

Assuming you are a data analyst / scientist at Target, you have been assigned the task of analyzing the given dataset to extract valuable insights and provide actionable recommendations.

What does 'good' look like?

1. Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:

1. Data type of all columns in the "customers" table.

```
Select Table_name,  
       Column_name,  
       Data_type  
From Target_SQL.INFORMATION_SCHEMA.COLUMNS  
Where Table_name = 'customers';
```

Output:-

Row	Table_name	Column_name	Data_type
1	customers	customer_id	STRING
2	customers	customer_unique_id	STRING
3	customers	customer_zip_code_prefix	INT64
4	customers	customer_city	STRING
5	customers	customer_state	STRING

Insights:- 1. There are a total of 5 Columns in the Output.

2. Out of Which 4 Columns have 'String' DataType and 1 have 'Integer' DataType.

2. Get the time range between which the orders were placed.

```
Select min(order_purchase_timestamp) as First_order_time,  
       max(order_purchase_timestamp) as Last_order_time  
From `Target_SQL.orders`;
```

Output:-

Row	First_order_time	Last_order_time
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC

Insights:- 1. Here output has One Row & 2 Columns.

2. First Column named 'First_order_time' Represents the First Order Placed Date.

3. Second Column named 'Last_order_time' Represents the Last Order Placed.

Recommendations:- 1. We have only 2 Years of Data.

3. Count the Cities & States of customers who ordered during the given period

```
Select Count(Distinct C.customer_state) as No_of_states ,  
       Count(Distinct C.customer_city) as No_of_cities  
From `Target_SQL.customers` C  
Inner Join `Target_SQL.orders` O  
On C.customer_id =O.customer_id  
Where O.order_purchase_timestamp between '2016-09-04' and '2018-10-17'  
Order by 1;
```

Output:-

No_of_states ▼	No_of_cities ▼
27	4119

Insights:- 1. We have one row in the Output.

Recommendations:- 1. In the Output Customer_City 'sao paulo' in State 'SP' has the highest number of Customers is 15540 .

2. So it is very much possible that most of the revenue of the company is from this city.

3. So the company should focus more on this City .

2. In-depth Exploration:

1. Is there a growing trend in the no. of orders placed over the past years?

```
Select extract(year from order_purchase_timestamp) as Year,
        Count( distinct order_id) as Total_orders,
From `Target_SQL.orders`
group by 1
order by 1;
```

Output:-

Row	Year ▼	Total_orders ▼
1	2016	329
2	2017	45101
3	2018	54011

- Insights:-
1. Yes, there is a growing trend in the number of orders placed over the past three years.
 2. From Year '2016' to Year '2017' there is a growth of around 13609 % which is unbelievable.
 3. From Year '2017' to Year '2018' there is a growth of around 20 % which is very less than previous year.

- Recommendations:-
1. From Year '2016' to '2017' there is a huge difference in Number of orders placed, but from Year '2017' to '2018' there is less difference in the number of orders placed.
 2. The company did very well in the year '2016' that's why in year '2017' there is a huge number of orders placed.

2. Can we see some kind of monthly seasonality in terms of the no. of orders being placed ?

```
Select  format_datetime('%Y-%m', order_purchase_timestamp) As Year_month,
        Count(order_id) AS total_orders_in_month
From `Target_SQL.orders`
Group by 1;
```

Output:-

Row	Year_month	total_orders_in...
1	2017-02	1780
2	2017-04	2404
3	2016-09	4
4	2016-10	324
5	2017-01	800
6	2017-03	2682
7	2017-05	3700
8	2017-06	3245
9	2017-07	4026
10	2017-08	4331

Insights:- 1. The above code retains the Total order placed in each year and in each month.

2. Each year in the Month of November the Number of Order Placed is Maximum.

Recommendations:- 1. In order to maintain the ease of credibility and readability, the data should be in Ordered form.

3. During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)

0-6 hrs : Dawn
7-12 hrs : Mornings
13-18 hrs : Afternoon
19-23 hrs : Night

```
Select Case When extract(Hour from order_purchase_timestamp) between 0 and 6 then 'Dawn'
            When extract(Hour from order_purchase_timestamp) between 7 and 12
                then 'Mornings'
            When extract(Hour from order_purchase_timestamp) between 13 and 18
                then 'Afternoon'
            When extract(Hour from order_purchase_timestamp) between 19 and 23
                then 'Night'
            End as time_of_the_day,
count(order_id) as total_orders
From `Target_SQL.orders`
group by 1
order by 2 desc;
```

Output:-

Row	time_of_the_day	total_orders
1	Afternoon	38135
2	Night	28331
3	Mornings	27733
4	Dawn	5242

- Insights:-
1. According to the Output, Highest Number of Order Placed is in 'Afternoon' between '13:00 Hour' to '18:00 Hour'.
 2. The Second Highest Number of Order Placed is in 'Night' between '19:00 Hour' to '23:00 Hour'.

- Recommendations:-
1. We can easily figure out that the number of orders placed between 'Afternoon' and 'Morning' and 'Afternoon' is a Huge Difference.
 2. It means customers are more active in 'Afternoon' than customers in 'Morning'.

3. Evolution of E-commerce orders in the Brazil region:

1. Get the month on month no. of orders placed in each state

```
Select  format_date('%Y-%m', order_purchase_timestamp) AS order_month,
        C.customer_state,
        Count(O.order_id) AS number_of_orders
From `Target_SQL.orders` O
Inner Join `Target_SQL.customers` C
On O.customer_id=C.customer_id
Group by 1,2 ;
```

Output:-

Row	order_month	customer_state	number_of_orders
1	2017-02	MG	259
2	2017-04	SP	908
3	2016-09	SP	2
4	2016-10	CE	8
5	2016-10	SP	113
6	2016-09	RS	1
7	2016-10	RJ	56
8	2016-10	MT	3
9	2016-10	GO	9
10	2016-10	SC	11

- Insights:- 1. In the output We got 565 Rows among each Month of every year along with the state.
2. This data helps out to analyze the geolocations of Number of orders placed in each state order by Year and month

Recommendations:- 1. From this data we can check whether most of the company orders come from , which directly play in the role of company revenue.

2. Obviously if we have the state name where most of the orders are placed, of course , there will be most of the customer traffic.This helps us to manage customers easily.

2. How are the customers distributed across all the states

```
Select customer_state,  
        Count(customer_id) as No_of_customers  
From `Target_SQL.customers`  
Group by 1;
```

Output:-

Row	customer_state	No_of_customers
1	AC	81
2	AL	413
3	AM	148
4	AP	68
5	BA	3380
6	CE	1336
7	DF	2140
8	ES	2033
9	GO	2020
10	MA	747

Insights:- 1. The Output for this is the Number of Customers In each State.
2. There are a total 27 States in which customers are distributed.

Recommendations:- 1. From the above data we can conclude the Number of Customers in Each State.

2. Which Helps us Significantly in Traffic Management.

4. Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

1. Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).

You can use the "payment_value" column in the payments table to get the cost of orders.

```
With Cost_in_2017 as (Select  sum(p.payment_value) AS total_cost_2017
                        From `Target_SQL.orders` O
                        Inner Join `Target_SQL.payments` P
                        On O.order_id = P.order_id
                        Where Extract (Year From O.order_purchase_timestamp)=2017
                        and Extract(Month From O.order_purchase_timestamp) between
                        1 AND 8 ),
```

```
Cost_in_2018 as (Select sum(p.payment_value) AS total_cost_2018
                    From `Target_SQL.orders` O
                    Inner Join `Target_SQL.payments` P
                    On O.order_id = P.order_id
                    Where Extract (Year From O.order_purchase_timestamp) = 2018
                    and Extract(Month From O.order_purchase_timestamp) between 1
                    AND 8 )
```

```
Select round((((Cost_in_2018.total_cost_2018 - Cost_in_2017.total_cost_2017) /
Cost_in_2017.total_cost_2017) * 100,2) as Percent_Increases
From Cost_in_2017, Cost_in_2018;
```

Output:-

Row	Percent_Increases
1	136.98

- Insights:-
1. The above code gives the Percentage Increase in cost of orders from year 2017 to 2018 (From month of Jan- Aug only)
 2. The above output is rounded off to 2 decimal for better readability.

Recommendations:- 1. With the above output we can figure out that company is growing as increase in Cost of Orders from 2017 to 2018.

2. Calculate the Total & Average value of order price for each state.

```
Select distinct C.customer_state,
               sum(OI.price) as Total_Value,
               Avg(OI.price) as Avg_order_price

From `Target_SQL.customers` C
Inner Join `Target_SQL.orders` O
On C.customer_id=O.customer_id
Inner Join `Target_SQL.order_items` OI
On O.order_id=OI.order_id
Group by 1
```

Output:-

Row	customer_state	Total_Value	Avg_order_price
1	AC	15982.94999999...	173.7277173913...
2	AL	80314.80999999...	180.8892117117...
3	AM	22356.84000000...	135.4960000000...
4	AP	13474.29999999...	164.3207317073...
5	BA	511349.9900000...	134.6012082126...
6	CE	227254.7099999...	153.7582611637...
7	DF	302603.9399999...	125.7705486284...
8	ES	275037.3099999...	121.9137012411...
9	GO	294591.9499999...	126.2717316759...
10	MA	119648.2199999...	145.2041504854...

Insights:- 1. In the above Output ,First Column represents the revenue generated from orders in each State.

2. The Second Column represents the Average order value of a customer in each State.

Recommendations:- 1. From the above Output we can conclude that which state has the highest or lowest Avg order Price that helps to figure out the Valuable state from which most of the orders placed and most of the Revenue comes from these states.

3. Calculate the Total & Average value of order freight for each state.

```
Select distinct C.customer_state,
      Round(sum(OI.freight_value),2) as Total_freight_value,
      Round(Avg(OI.freight_value),2) as Avg_freight_value

From `Target_SQL.customers` C
Inner Join `Target_SQL.orders` O
On C.customer_id=O.customer_id

Inner Join `Target_SQL.order_items` OI
On O.order_id=OI.order_id
Group by 1
```

Output:-

Row	customer_state	Total_freight_value	Avg_freight_value
1	AC	3686.75	40.07
2	AL	15914.59	35.84
3	AM	5478.89	33.21
4	AP	2788.5	34.01
5	BA	100156.68	26.36
6	CE	48351.59	32.71
7	DF	50625.5	21.04
8	ES	49764.6	22.06
9	GO	53114.98	22.77
10	MA	31523.77	38.26

Insights:- 1. In the above Output , First Column represents the revenue generated from Order Freight in each State.

2. The Second Column represents the Average freight value of Orders in each State.

3. In the Both Columns the value is Rounded to 2 decimal places for better understanding.

Recommendations:- 1. From the above Output we can conclude that which state has highest or lowest freight order value that helps to figure out the Valuable state from which most of the orders placed and most of the Revenue comes from these states.

2. Freight Value totally depends on order type and location.

5. Analysis based on sales, freight and delivery time.

1. Find the no. of days taken to deliver each order from the order's purchase date as delivery time.

Also, calculate the difference (in days) between the estimated & actual delivery date of an order.

Do this in a single query.

You can calculate the delivery time and the difference between the estimated & actual delivery date using the given formula:

`#. time_to_deliver = order_delivered_customer_date -
order_purchase_timestamp`

`#. diff_estimated_delivery = order_delivered_customer_date -
order_estimated_delivery_date`

```

Select order_id,
       order_purchase_timestamp,
       order_delivered_customer_date,
       order_estimated_delivery_date,
       date_diff(order_delivered_customer_date , order_purchase_timestamp,
       day) as Time_to_deliver,
       date_diff(order_delivered_customer_date,
       order_estimated_delivery_date, day) as diff_estimated_delivery
From `Target_SQL.orders`

```

Output:-

Row	order_id	order_purchase_timestamp	order_delivered_customer_date	order_estimated_delivery_date	Time_to_deliver	diff_estimated_d...
1	65d1e226dfaeb8cdc42f665422...	2016-10-03 21:01:41 UTC	2016-11-08 10:58:34 UTC	2016-11-25 00:00:00 UTC	35	-16
2	2c45c33d2f9cb8ff8b1c86cc28...	2016-10-09 15:39:56 UTC	2016-11-09 14:53:50 UTC	2016-12-08 00:00:00 UTC	30	-28
3	1950d777989f6a877539f53795...	2018-02-19 19:48:52 UTC	2018-03-21 22:03:51 UTC	2018-03-09 00:00:00 UTC	30	12
4	bfbdf0f9bdef84302105ad712db...	2016-09-15 12:16:38 UTC	2016-11-09 07:47:38 UTC	2016-10-04 00:00:00 UTC	54	36
5	98974b076b01553d49ee64679...	2016-10-04 15:44:44 UTC	2016-11-17 08:38:57 UTC	2016-11-24 00:00:00 UTC	43	-6
6	c4b41c36dd589e901f6879f25a...	2016-10-04 12:53:17 UTC	2016-11-09 13:37:38 UTC	2016-11-24 00:00:00 UTC	36	-14
7	d2292ff2201e74c5db154d1b7a...	2016-10-04 16:05:29 UTC	2016-11-03 10:08:51 UTC	2016-11-24 00:00:00 UTC	29	-20
8	95e01270fcb9e986342340010...	2016-10-04 18:52:56 UTC	2016-11-04 17:54:00 UTC	2016-11-24 00:00:00 UTC	30	-19
9	ed8c7b1b3eb256c70ce0c7423...	2016-10-04 09:59:03 UTC	2016-11-18 08:51:07 UTC	2016-11-24 00:00:00 UTC	44	-5
10	5cc475c7c03290048eb2e742c...	2016-10-04 21:54:52 UTC	2016-12-12 20:31:54 UTC	2016-11-24 00:00:00 UTC	68	18

Insights:- 1. In the Output we have majorly 2 important Columns:-

1. The first column named 'Time_to_deliver' represents the actual time taken to deliver the order to the customer.
2. While the second column named 'diff_estimated_delivery' represents the estimation delivery time which was previously given to the customer for Reference.
3. Here in the second column 'diff_estimated_delivery' some values are positive and some values are negative.
4. The negative value represents the time taken for actual delivery to customer is greater than estimated delivery time , while positive value defines that actual delivery happens earlier as compared to estimated delivery time.

Recommendations:- 1. This helps us to Identify whether delivery done in expected time or not, if not where is the issues, like

Is delivery delays due to a single product?
Is delivery delays due to the freight carrier ?
Is delivery delays due to the wrong address given by the customer?

We can get our answers from the above data easily.

2. Find out the top 5 states with the highest & lowest average freight value.

```
With StateAvgFreight as
(select C.customer_state as State,
      avg(OI.freight_value) as average_freight_value
From `Target_SQL.orders` O
Inner join `Target_SQL.customers` C
On O.customer_id=C.customer_id
```

```
Inner join `Target_SQL.order_items` OI
On OI.order_id=O.order_id
group by 1 ),
```

```
HighestFreight as
(select State,
      average_freight_value,
      'Highest' as category
From StateAvgFreight
order by 2 desc
limit 5 ),
```

```
LowestFreight as
(select State,
      average_freight_value,
      'Lowest' as category
From StateAvgFreight
order by 2 asc
limit 5 )
```

```
select *
From HighestFreight
union all
select *
```

```
from LowestFreight
order by 3 desc ,2 asc;
```

Output:-

Row	State ▼	average_freight_v...	category ▼
1	SP	15.14727539041...	Lowest
2	PR	20.53165156794...	Lowest
3	MG	20.63016680630...	Lowest
4	RJ	20.96092393168...	Lowest
5	DF	21.04135494596...	Lowest
6	PI	39.14797047970...	Highest
7	AC	40.07336956521...	Highest
8	RO	41.06971223021...	Highest
9	PB	42.72380398671...	Highest
10	RR	42.98442307692...	Highest

Insights:-

1. In output we have a total of 10 rows and Three columns.
2. The first Column represents the State for which we are looking for.
3. Second column represents the average freight value for each state.
4. Third column represents the category which has the lowest and Highest average freight value.

Recommendations:-

1. From this data we can get information about which state has the highest and lowest average_freight_vale.

3. Find out the top 5 states with the highest & lowest average delivery time.

```
With StateAvgDelivery as
(select C.customer_state as State, Round(avg(date_diff(
0.order_delivered_customer_date , 0.order_purchase_timestamp,
day)),2) as
```

```
average_delivery_time
From `Target_SQL.orders` O
Inner join `Target_SQL.customers` C
On O.customer_id=C.customer_id
```

```
Inner join `Target_SQL.order_items` OI
On OI.order_id=O.order_id
group by 1 ),
```

```
HighestDeliveryTime as
(
select State,
        average_delivery_time,
        'Highest' as category
From StateAvgDelivery
order by 2 desc
limit 5
),
```

```
LowestDeliveryTime as
(
select State,
        average_delivery_time,
        'Lowest' as category
From StateAvgDelivery
order by 2 asc
limit 5
```

```
)
select *
From HighestDeliveryTime
union all
select *
from LowestDeliveryTime
order by 3 asc ,2 asc;
```


Output:

Row	State	average_delivery_...	category
1	PA	23.3	Highest
2	AL	23.99	Highest
3	AM	25.96	Highest
4	AP	27.75	Highest
5	RR	27.83	Highest
6	SP	8.26	Lowest
7	PR	11.48	Lowest
8	MG	11.52	Lowest
9	DF	12.5	Lowest
10	SC	14.52	Lowest

- Insights:-
1. In output we have a total of 10 rows and Three columns.
 2. The first column represents the State for which we are looking for.
 3. The second column represents the average delivery time for each state.
 - 4.Third column represents the category which has the lowest and Highest average delivery time.

Recommendations:- 1. From this data we can get information about which state has the highest and lowest average_delivery_time.

4. Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.

You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

```
With StateAvgDelivery as
(select C.customer_state as State,
```

```

avg(date_diff(0.order_estimated_delivery_date,
0.order_delivered_customer_date, day)) as average_delivery_time
From `Target_SQL.orders` O
Inner join `Target_SQL.customers` C
On O.customer_id=C.customer_id

Inner join `Target_SQL.order_items` OI
On OI.order_id=O.order_id
group by 1 ),

```

```

HighestDeliveryTime as
( select State,
        average_delivery_time,
        'Highest' as category
  From StateAvgDelivery
 order by 2 desc
 limit 5 ),

```

```

LowestDeliveryTime as
( select State,
        average_delivery_time,
        'Lowest' as category
  From StateAvgDelivery
 order by 2 asc
 limit 5 )

```

```

Select *
From HighestDeliveryTime
union all
select *
from LowestDeliveryTime
order by 3 asc ,2 asc;

```

Output:-

Row	State	average_delivery_time	category
1	RR	17.434782608695649	Highest
2	AP	17.444444444444446	Highest
3	AM	18.975460122699381	Highest
4	RO	19.080586080586077	Highest
5	AC	20.010989010989007	Highest
6	AL	7.9765807962529216	Lowest
7	MA	9.1100000000000048	Lowest
8	SE	9.1653333333333347	Lowest
9	ES	9.768539325842724	Lowest
10	BA	10.119467825142548	Lowest

- Insights:-
1. In output we have a total of 10 rows and Three columns.
 2. The first column represents the State for which we are looking for.
 3. The second column represents the average delivery time for each state.
 4. The third column represents the category which has the lowest and Highest average delivery time.
 5. Highest Value represents how fast the delivery happened .

- Recommendations:-
1. From this data we can get information about which state has the highest and lowest average_delivery_time.
 2. Here are two loop holes:-
 1. Delivery time means time difference between Order purchase date and order delivery date So it is considered the delivery time.
 2. To know the weather how fast the actual delivery is so it is the difference between Estimated delivery time and Actual delivery delivery time.

6. Analysis based on the payments:

1. Find the month on month no. of orders placed using different payment types.

```
Select date_trunc( O.order_purchase_timestamp , Month) as Order_month,
       P.payment_type,
       Count(O.order_id) as number_of_orders
From `Target_SQL.orders` O
Inner join `Target_SQL.payments` P
On O.order_id=P.order_id
Group by 1,2
Order by 1,2;
```

Output:-

Row	order_month	payment_type	number_of_orders
1	2016-09-01 00:00:00 UTC	credit_card	3
2	2016-10-01 00:00:00 UTC	UPI	63
3	2016-10-01 00:00:00 UTC	credit_card	254
4	2016-10-01 00:00:00 UTC	debit_card	2
5	2016-10-01 00:00:00 UTC	voucher	23
6	2016-12-01 00:00:00 UTC	credit_card	1
7	2017-01-01 00:00:00 UTC	UPI	197
8	2017-01-01 00:00:00 UTC	credit_card	583
9	2017-01-01 00:00:00 UTC	debit_card	9
10	2017-01-01 00:00:00 UTC	voucher	61

Insights:- 1. Here we know which payment method is very popular among customers.
2. From the above data we can conclude that 'Credit Cards' are very popular among customers.

Recommendations:- 1. Due to convenience of Credit card Such as rewards, Points and Cashbacks on it customers widely used it.

2. Find the no. of orders placed on the basis of the payment installments that have been paid.

```
Select payment_installments,  
       Count(distinct order_id) as number_of_orders  
From `Target_SQL.payments`  
Group by 1  
Order by 1;
```

Output:-

Row	payment_installm...	number_of_orders
1	0	2
2	1	49060
3	2	12389
4	3	10443
5	4	7088
6	5	5234
7	6	3916
8	7	1623
9	8	4253
10	9	644

Insights:- 1. Most of the Customer does not avail EMI facility due to Higher Interest Rate or Inconvenient.

2. Some customers used the Installment facility due to Higher Order Value.

Recommendations:- 1. Offer the No cost EMI feature to customers for Payment method.

2. Use cashback or voucher scheme to promote the EMI facility.